SQL

Stop Memorizing 100 SQL Queries. These 10 Are Enough

10 queries that cover most of what data analysts write at work and get asked in interviews. Each one with the problem, the query and why it matters.

INSIDE THIS PDF · 9 PAGES

- Practise on this table
- Query 1 and 2
- Query 3 and 4
- Query 5 and 6
- Query 7 and 8
- Query 9
- Query 10

@thedataguy16

Practise on this table

All 10 queries use these two tables. Create them once and run every query yourself.

```
orders(order_id, customer_id, order_date,
       amount, status)

customers(customer_id, name, city,
          signup_date)

employees(emp_id, name, dept,
          salary, manager_id)
```

How to use this PDF

Read the problem, try writing the query yourself for 5 minutes, then compare.

Where to practise free

SQLite online, DB Fiddle, or the free tier of any cloud warehouse.

@thedataguy16

Query 1 and 2

Aggregation and joins. The base of every report.

1. Customers with more than 5 orders

WHERE filters rows before grouping. HAVING filters the groups after.

```
SELECT customer_id, COUNT(*) AS orders
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

2. Customers who never ordered

LEFT JOIN keeps every customer. A NULL on the right side means no match.

```
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.customer_id
WHERE o.order_id IS NULL;
```

@thedataguy16

Query 3 and 4

Window functions. The most asked topic in data analyst SQL rounds.

3. Top 3 earners in each department

DENSE_RANK gives ties the same rank and does not skip numbers.

```
SELECT * FROM (
  SELECT name, dept, salary,
    DENSE_RANK() OVER (PARTITION BY dept
      ORDER BY salary DESC) AS rk
  FROM employees) t
WHERE rk <= 3;
```

4. Running total of revenue by day

Each row adds everything before it. Used in every growth dashboard.

```
SELECT order_date, SUM(amount) AS day_rev,
  SUM(SUM(amount)) OVER (ORDER BY order_date)
    AS running_rev
FROM orders
GROUP BY order_date;
```

@thedataguy16

Query 5 and 6

Comparing periods and cleaning duplicates.

5. Month on month revenue growth %

LAG reads the previous row. Divide by it to get growth.

```
WITH m AS (
  SELECT DATE_TRUNC('month', order_date) AS mon,
         SUM(amount) AS rev
  FROM orders GROUP BY 1)
SELECT mon, rev,
  ROUND(100.0 * (rev - LAG(rev) OVER (ORDER BY
mon))
    / LAG(rev) OVER (ORDER BY mon), 1) AS
mom_pct
FROM m;
```

6. Keep only the latest row per customer

ROW_NUMBER numbers each row in a group. Keep number 1.

```
SELECT * FROM (
  SELECT o.*, ROW_NUMBER() OVER (
    PARTITION BY customer_id
    ORDER BY order_date DESC) AS rn
  FROM orders o) t
WHERE rn = 1;
```

@thedataguy16

Query 7 and 8

Turning rows into columns, and the classic interview opener.

7. Delivered vs returned orders by city

CASE inside SUM counts only the rows that match. A pivot without PIVOT.

```
SELECT c.city,
  SUM(CASE WHEN o.status = 'Delivered' THEN 1
ELSE 0 END) AS delivered,
  SUM(CASE WHEN o.status = 'Returned'  THEN 1
ELSE 0 END) AS returned
FROM orders o
JOIN customers c USING (customer_id)
GROUP BY c.city;
```

8. Second highest salary

Works even when two people share the top salary.

```
SELECT DISTINCT salary FROM (
  SELECT salary,
    DENSE_RANK() OVER (ORDER BY salary DESC) AS
rk
  FROM employees) t
WHERE rk = 2;
```

@thedataguy16

Query 9

Streaks. Asked to check if you can think in patterns, not just syntax.

9. Customers who ordered 3 days in a row

Date minus row number stays constant across a streak. Group by it.

```
WITH d AS (
  SELECT DISTINCT customer_id, order_date FROM
orders),
g AS (
  SELECT customer_id, order_date,
    order_date - CAST(ROW_NUMBER() OVER (
      PARTITION BY customer_id ORDER BY
order_date)
      AS INT) AS grp
  FROM d)
SELECT customer_id FROM g
GROUP BY customer_id, grp
HAVING COUNT(*) >= 3;
```

Why it works: dates 3, 4, 5 have row numbers 1, 2, 3. Date minus row number is 2 for all three, so they land in one group.

@thedataguy16

Query 10

Retention. The first metric any product or growth team asks an analyst for.

10. Month 1 to month 2 retention

Of customers who signed up in a month, how many ordered the next month.

```
SELECT DATE_TRUNC('month', c.signup_date) AS
cohort,
  COUNT(DISTINCT c.customer_id) AS signed_up,
  COUNT(DISTINCT o.customer_id) AS came_back
FROM customers c
LEFT JOIN orders o
  ON o.customer_id = c.customer_id
 AND DATE_TRUNC('month', o.order_date) =
     DATE_TRUNC('month', c.signup_date) +
INTERVAL '1 month'
GROUP BY 1;
```

Retention % = came_back ÷ signed_up × 100. Present it as a cohort table, one row per signup month.

@thedataguy16

What these 10 cover

Learn the pattern behind each query, not the exact text. Most interview questions are one of these in a new costume.

- **Filter groups**
  Query 1 (HAVING)
- **Find missing records**
  Query 2 (anti join)
- **Rank within groups**
  Queries 3 and 8
- **Running and period comparisons**
  Queries 4 and 5
- **Deduplicate**
  Query 6
- **Pivot**
  Query 7
- **Streaks and retention**
  Queries 9 and 10

Syntax note: date functions differ by database. DATE_TRUNC and INTERVAL are PostgreSQL. In MySQL use DATE_FORMAT and DATE_ADD.

@thedataguy16